



Trabajo Práctico
Análisis de Texto

Fecha de entrega: 09/04/2026

Josue Leonel Gatica Odató 178921

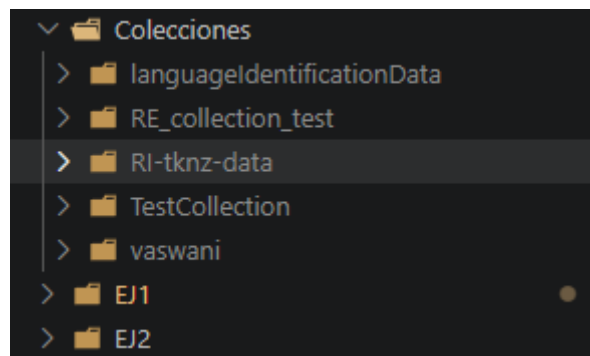
Josuegaticaodato@gmail.com





Aclaración importante sobre colecciones:

Para un correcto manejo de las colecciones en cada ejercicio, los archivos .py tomando las colecciones por defecto en una carpeta llamada “Colecciones”, ubicada a la par con todas las carpetas de los ejercicios.



En ejercicios donde no se indica el pasaje del directorio donde se encuentran los documentos como parámetro (por ejemplo, el EJ1), este toma por defecto la carpeta indicada con el nombre de la colección que se solicitó analizar para ese ejercicio.

Herramienta de IA

El modelo de lenguaje de inteligencia artificial utilizado para el trabajo fue ChatGPT



1) Utilizando la colección para debugging provista por el equipo docente escriba un programa que realice el análisis léxico sobre la misma y extraiga la siguiente información:

- Lista de términos y su DF
- Cantidad de *tokens*
- Cantidad de términos
- Cantidad de documentos procesados

Compare los resultados obtenidos por su programa con los que figuran en el archivo *collection_data.json* que contiene los metadatos de la colección.

Primero, para generar el analizador léxico, recuperamos los tokens que contiene cada texto, pasando toda a minúsculas y eliminando símbolos.

```
1 def normalizar(texto):
2     texto = texto.lower() # Minusculas
3     texto = re.sub(r'^a-záéíóúñ\s', ' ', texto) # Eliminar simbolos
4     tokens = texto.split() # Split por espacio
5     return tokens
```

Luego, realizamos la construcción del índice para cada documento, para obtener la cantidad de apariciones de cada token en ese documento..

```
1 for archivo in archivos:
2     documentos += 1
3
4     # Nombre del archivo a numero
5     doc_id = int(archivo.replace("doc", "").replace(".txt", ""))
6
7     print(os.path.join(ruta, archivo))
8     with open(os.path.join(ruta, archivo), "r", encoding="utf-8") as f:
9         texto = f.read()
10
11     # Obtengo los tokens
12     tokens = normalizar(texto)
13     total_tokens += len(tokens)
14
15     # Construccion del indice
16     for token in tokens:
17         indice[token][doc_id] += 1
18
19     # Agregar palabras al vocabulario (como es conjunto no repite, obteniendo los terminos unicos)
20     vocabulario.update(tokens)
```

Por último, generamos los arreglos correspondientes para docids y frecuencias.

```
1 for termino, docs in indice.items():
2     # Ordenar documentos por docid
3     pares_ordenados = sorted(docs.items())
4
5     docids = [doc for doc, _ in pares_ordenados]
6     freqs = [freq for _, freq in pares_ordenados]
```

Ya con esto, se exporta en formato JSON, siguiente la estructura solicitada.

```
{
  "data": [
    {
      "term": "gato",
      "docid": [...],
      "freq": [...],
      "df": 400
    },
    { ... }
  ],
  "statistics": {
    "N": 10000,
    "num_terms": 20,
    "num_tokens": 793254
  }
}
```

Analizándolo en detalle con el collection_data.json provisto por el equipo docente, solamente se difiere en orden en el que se muestran los términos, ya que no siguen un patrón claro para poder ordenarlo de la siguiente manera y que ambas JSON sean iguales.

Luego, en cuanto a contenido, son exactamente iguales, garantizado la correcta realizando del ejercicio.

Utilización de IA:

Utilizando la herramienta de inteligencia artificial ChatGPT, se realizó lo siguiente:

- Obtención de una expresión regular para la eliminación de símbolos dentro del texto
- Explicaciones con ejemplos para manejos de índices invertidos y pares ordenados
- Construcción de rutas para lectura correcta de archivos

La validez de la respuesta se verifico en base a pruebas manuales sobre cada fragmento realizado por el chat, validando que los resultados sean los correcto.

Ayudo en el proceso de aprendizaje mediante explicaciones sencillas con ejemplos y pequeños códigos de uso general que serán utilizados en los demás puntos



2) Escriba un programa que realice análisis léxico sobre la colección **RI-tnkz-data**. El programa debe recibir como parámetros el directorio donde se encuentran los documentos y un argumento que indica si se deben eliminar las palabras vacías (y en tal caso, el nombre del archivo que las contiene). Defina, además, una longitud mínima y máxima para los términos. Como salida, el programa debe generar:

- Un archivo (`terminos.txt`) con la lista de términos a indexar (ordenado), su frecuencia en la colección y su DF (*Document Frequency*).

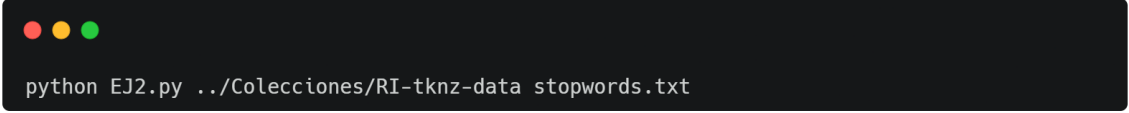
Formato de salida: `$<termino> [ESP] <CF> [ESP] <DF>$`.

Ejemplo:

```
casa 238 3
perro 644 6
...
zorro 12 1
```

- Un segundo archivo (`estadisticas.txt`) con los siguientes datos (un ítem por línea y separados por espacio cuando sean más de un valor):
 - Cantidad de documentos procesados.
 - Cantidad de tokens y términos extraídos.
 - Promedio de tokens y términos de los documentos.
 - Largo promedio de un término.
 - Cantidad de *tokens* y términos del documento más corto y del más largo.
 - Cantidad de términos que aparecen sólo 1 vez en la colección.
- Un tercer archivo (`frecuencias.txt`), con:
 - La lista de los 10 términos más frecuentes y su CF (*Collection Frequency*). Un término por línea.
 - La lista de los 10 términos menos frecuentes y su CF. Un término por línea.

Ejecución del programa:



```
python EJ2.py ../Colecciones/RI-tnkz-data stopwords.txt
```

Construcción del analizador léxico

El tokenizer utilizado para realiza lo siguiente:

- Eliminar espacios vacíos
- Pasar todos a minúsculas
- Permitir solo términos que estén entre los valores mínimos y máximos
- Eliminar stopwords (Si se envió el archivo)



1) Obtención del archivo términos.txt

La salida del archivo términos.txt queda de la siguiente manera (mostrando los primeros 10 términos aproximadamente):

```
universidad 309 309
nacional 297 297
argentina 215 215
ar 200 200
edu 199 199
información 176 176
ciencias 164 164
investigación 163 163
facultad 159 159
inicio 146 146
...
```

Con stopwords, el termino mas frecuente es “universidad” con 309 apariciones.

2) Obtención del archivo estadísticas.txt

A continuación, se muestra el contenido del archivo estadísticas.txt

```
498
220971 27068
443.72 187.21
8.53
rince.unlam.edu.ar.txt 0 0
extensiondigital.fpsico.unr.edu.ar.txt 22395 7344
17215
```

Lo que puede observar en base a estas estadísticas es lo siguiente:

- La distinción notoria entre token y termino hablada en clase,
- Mas de la mitad de los términos extraídos aparecen solo una vez en todos los documentos,
- El archivo mas corto contiene solamente “r.i.n.c.e.” que, en base a nuestro tokenizer, no es considerado ni token ni termino. Esto se solucionara en el ejercicio 3

3) Obtención del archivo frecuencias.txt

El contenido de frecuencias.txt es el siguiente:

```
Los 10 terminos mas frecuentes y su CF (Collection Frequency):  
universidad 309  
nacional 297  
argentina 215  
ar 200  
edu 199  
información 176  
ciencias 164  
investigación 163  
facultad 159  
inicio 146  
  
Los 10 terminos menos frecuentes y su CF (Collection Frequency):  
detectando 1  
neutralizando 1  
educativastrámite 1  
tracción 1  
saspi 1  
sspi 1  
sel 1  
ddb 1  
sistemasgcyt 1  
target 1
```

En el caso de no aplicar stopwords, estas ocuparan los 10 primeros términos más frecuentes. Si es necesario llevar a cabo un correcto análisis, deben ser eliminadas.

Analizando los 10 primeros términos, puede verse como una gran cantidad de documentos están vinculados a información de universidades argentinas.

Luego, los 10 términos menos frecuentes se muestran como combinaciones de palabras (educativas y tramite, educativastraimte) o palabras que fueron nombradas solo una vez por el contexto del documento (el termino target puede haber sido incluido por error, ya que todo el contenido de los documentos esta en español).

3) Escriba un segundo *tokenizer* que implemente los criterios del artículo de Grefenstette y Tapanainen para definir qué es una “palabra” (o término) y cómo tratar números y signos de puntuación. En este caso su *tokenizer* deberá extraer y tratar como un único término:

- Abreviaturas tal cual están escritas (por ejemplo, Dr., Lic., S.A., etc.)
- Direcciones de correo electrónico y URLs.
- Números (por ejemplo, cantidades, teléfonos).
- Nombres propios (por ejemplo, Villa Carlos Paz, Manuel Belgrano, etc.)

Utilice la colección para *debugging* de expresiones regulares provista por el equipo docente para extraer y comparar la salida de su programa con los metadatos de la colección tal como lo realizó en el punto 1.

Por último, extraiga y almacene la misma información que en el punto 2 sobre la colección **RI-tknz-data** utilizando su nuevo *tokenizer*.

En base a los criterios del artículo de Grefenstette y Tapanainen, se construyó el siguiente Tokenizer:

```
token_especificos = [
    ('EMAIL', r"[a-zA-Z0-9][a-zA-Z0-9._%+\-]*@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}"),
    ('URL', r'(?:(https?|ftp?):\/\/[^\s<>"\`']+)',
    ('ABBREV_MULTI', r"(?:[A-Za-záéíóúüñÁÉÍÓÚÜÑ]{1,4}\.){1,}[A-Za-záéíóúüñÁÉÍÓÚÜÑ]{1,4}\.?"", # Abreviaturas S.A. o U.S.A.
    ('ABBREV', r"[A-Za-záéíóúüñÁÉÍÓÚÜÑ]{2,10}\.", # Abreviaturas Dr., Lic.
    ('PHONE', r"[+\\-]?\\d[\\d\\.\\-]*\\((?:%|°)?)", # Telefonos
    ('NUMBER', r"\\d+(?:[\\.\\d+]*'"),
    ('FECHA', r"\\d{4}[\\-\\/]\\d{1,2}[\\-\\/]\\d{1,2}[\\d{1,2}[\\-\\/]\\d{1,2}[\\-\\/]\\d{4}"",
    ('PROPER_NOUN', r"(?:[A-ZÁÉÍÓÚÜÑ][a-záéíóúüñ+)(?:\\s+(?!Sra\\b|Sr\\b|Dr\\b)[A-ZÁÉÍÓÚÜÑ][a-záéíóúüñ+)+)", # Nombres propios, con varias palabras con mayúscula
    ('WORD', r"[a-záéíóúüñA-ZÁÉÍÓÚÜÑ]{2,}"",
    ('PUNCT', r'[!\\?.,;:]'"),
    ('SIGLA', r"[A-ZÁÉÍÓÚÜÑ]{2,}"")
]

def nuevo_tokenizer(texto):
    tok_regex = '|'.join(f'(?P<{name}>{regex})' for name, regex in token_especificos)

    tokens = []

    for match in re.finditer(tok_regex, texto):
        kind = match.lastgroup
        value = match.group().strip()

        tokens.append(value)

    return tokens
```

El nuevo tokenizer ahora permite emails, abreviaturas, urls, números y nombres propios. Fue necesaria la construcción de una lista de tuplas para cada token específico, vinculándole la expresión regular que más se adecua a ese término.



Se verifico con el collection_data.json que los términos sean tomados de una forma correcta, siguiendo con el análisis sobre la colección RI-tnkz-data

```
python EJ3.py ../Colecciones/RI-tnkz-data stopwords.txt
```

1) Obtención del archivo estadísticas.txt

```
498
236044 34312
473.98 208.63
8.29
rince.unlam.edu.ar.txt 1 1
extensiondigital.fpsico.unr.edu.ar.txt 23263 8274
23044
```

Comparando estas estadísticas con las del ejercicio 2, llegamos a lo siguiente:

- El nuevo tokenizer me posibilita tener más términos y tokens, ya que tomamos en cuenta expresiones regulares particulares para ciertos tokens (emails, urls, números, etc.).
- El único termino/token que estaba presente en el documento mas corto ahora si es contador por el nuevo tokenizer.
- Existe mas de 7000 nuevos términos que aparecen una vez en la colección (antes eran 17000)

2) Obtención del archivo frecuencias.txt

```
Los 10 terminos mas frecuentes y su CF (Collection Frequency):  
universidad 307  
nacional 295  
argentina 194  
información 172  
ciencias 164  
investigación 162  
facultad 158  
2012 149  
inicio 146  
institucional 136  
  
Los 10 terminos menos frecuentes y su CF (Collection Frequency):  
plegado 1  
target 1  
3725 1  
sistemasgcyt 1  
3727 1  
3697 1  
sel 1  
ddb 1  
saspi 1  
sspi 1
```

De los 10 términos mas frecuentes, podemos descartar la aparición del numero 2012, que no estaba presente en el análisis anterior, debido a que el tokenizer no lo consideraba valido. Esto nos ayuda mucho mas a la tarea de recuperación, ya que identificamos que gran parte de los documentos mencionan ese año.

En lo que involucra a los 10 términos menos frecuentes, en comparativa con el ejercicio anterior, aparecen números, que con nuestro nuevo tokenizer son considerados términos.

Utilización de IA:

Fue necesaria la intervención de la herramienta de IA para poder construir las expresiones regulares correctas basadas en el artículo, con la finalidad de que se realice una correcta tokenizacion sobre los documentos. La comprobación para ello fue el archivo collections_data.json provisto por el equipo docente.

- 4) A partir del programa del ejercicio 1, incluya un proceso de *stemming*. Luego de modificar su programa, corra nuevamente el proceso del ejercicio 2 y analice los cambios en la colección. ¿Qué implica este resultado? Busque ejemplos de pares de términos que tienen la misma raíz pero que el *stemmer* los trató diferente y términos que son diferentes y se los trató igual.

Utilizando la librería de NLTK, utilizamos SnowBallStemmer para hacer el stemming:

```
# Stemming usando SnowballStemmer, libreria de nltk
def stemming(terminos):
    stemmer = SnowballStemmer("spanish")
    return [stemmer.stem(termino) for termino in terminos]
```

Consulta a la IA sobre cómo funciona SnowBallStemmer:

SnowBallStemmer reduce palabras a su raíz aplicando reglas sobre sufijos, específicas para cada idioma. Implementa una evolución del algoritmo de Martin Porter.

Ej. Si la palabra es Nacionalmente

1. Sufijo mente. Como no está en una región valida se borra
Nacionalmente a Nacional
2. Detecta -al, como esta en región valida se deja
Queda finalmente nacional

Este solo soporta el idioma que se le envié como parámetro

Ejecución del programa

```
python EJ4.py ../Colecciones/RI-tnkz-data stopwords.txt
```

1) Obtención del archivo términos.txt

```
univers 379 317
nacional 356 306
public 355 166
inform 355 213
estudi 345 187
investig 332 187
academ 331 176
```

Puede verse como la palabra “Universidad” fue pasada a “Univers”, aplicando las reglas de Stemming de la librería..

2) Obtención del archivo estadísticas.txt

```
498
220971 16494
443.72 187.21
7.13
rince.unlam.edu.ar.txt 0 0
extensiondigital.fpsico.unr.edu.ar.txt 22395 7344
9853
```

Se puede notar lo siguiente:

- La cantidad de tokens sigue igual, pero la cantidad de términos bajo de 27000 a 16000 aproximadamente, alrededor de un tercio de los términos fueron pasados a su raíz morfológica.
- Eliminar una tercia parte de los términos hizo que baje el largo promedio de estos, de 8.53 a 7.13 (una letra menos)
- La cantidad de términos que aparecen una vez también bajo de 15000 a 9800.

3) Obtención del archivo frecuencias.txt

```
Los 10 terminos mas frecuentes y su CF (Collection Frequency):
univers 379
nacional 356
public 355
inform 355
estudi 345
investig 332
academ 331
desarroll 326
ingres 304
educ 304

Los 10 terminos menos frecuentes y su CF (Collection Frequency):
ligotecni 1
publicaseudem 1
colmen 1
utilesgal 1
fisur 1
target 1
saspi 1
ddb 1
sspi 1
sistemasgcyt 1
```

Los términos más frecuentes no cambiaron, y los términos menos frecuentes fueron afectados por el stemming, aunque las reglas aplicadas no hicieron que se reduzca drásticamente la palabra. Esto último puede deberse a que estamos tratando con palabras combinadas (españolversion por ejemplo, debería de ser tratada por separado, pero aparece junta en uno de los documentos)



Ejemplo misma raíz pero el stemmer los trata diferente

Los siguientes 2 términos:

- Rápida
- Rapidez

El stemmer los transformo en:

- Rapid
- Rapidez

Deberían mapearse como Rapid ambos supuestamente.

Ejemplo términos diferentes pero el stemmer los trata igual

Los siguientes 2 términos:

- Universal
- Universidad

Fueron pasados a Univers. Ambos términos no son lo mismo, pero el stemmer lo toma como iguales

Utilización de IA:

- Explicación del funcionamiento SnowBallStemmer, junto con ejemplos
- Ejemplos de pares de términos que podrían estar incluidos dentro de los documentos donde se llevó a cabo el análisis, para poder explicar correctamente los problemas del stemmer

- 5) Sobre la colección Vaswani, ejecute los *stemmers* Porter y Lancaster provistos en el módulo `nltk.stem`. Compare: cantidad de tokens únicos resultantes, resultado 1 a 1 y tiempo de ejecución para toda la colección. ¿Qué conclusiones puede obtener de la ejecución de uno y otro?

Resultados obtenidos:

Corriendo ambos stemmers sobre la colección Vaswani, se obtuvieron los siguientes resultados.

```
Comparacion de Stemmers: Porter vs. Lancaster
-----
Tokens: 479163
Vocabulario: 12189
-----
Cantidad de tokens unicos resultantes:
Porter: 7993
Lancaster: 6877
-----
Tiempo de ejecucion:
Porter: 5.43 seg.
Lancaster: 4.29 seg.
```

	Porter	Lancaster
Tokens Únicos	7993	6877
Tiempo de ejecución	5.43s	4.29s

Viendo en detalle la cantidad de tokens únicos que genero cada uno, puede verse que Porter genero más de 1000 tokens adicionales que Lancaster, esto se debe a que prevalece un enfoque conservador en este stemmer, tratando de no recortar tan bruscamente las palabras y garantizar que las raíces se puedan interpretar mejor. En cambio, Lancaster tiene reglas más agresivas en su algoritmo, reduciendo una gran cantidad de palabras, pero trae consigo el problema de deformar demasiado estas.

En velocidad, es evidente que el enfoque brusco de Lancaster garantiza que el algoritmo realice más rápidamente el stemming, mientras que Porter le toma un tiempo más aplicar sus reglas que favorecen la semántica.



6) Escriba un programa que realice la identificación del lenguaje de un texto a partir de un conjunto de entrenamiento. Pruebe dos métodos sencillos:

- Uno basado en la distribución de la frecuencia de las letras.
- El segundo, basado en calcular la probabilidad de que una letra x preceda a otra letra y (calcule una matriz de probabilidades con todas las combinaciones).

Compare los resultados contra el módulo Python `langdetect` y la solución provista.

Existe 3 archivos:

- `Langdetect-library.py`
Utilizando la librería `langdetect`, realiza la identificación de idiomas
- `EJ6.py`
Utiliza el método de distribución de frecuencias de las letras para identificar idiomas
- `EJ6Matriz.py`
Utiliza el cálculo de la probabilidad de las letras precedentes para identificar idiomas

Ambos métodos se compararon con la solución provista y el resultado en `langdetect`

Langdetect

```
Resultados - Distribucion de la frecuencia de las letras
---- COMPARACION CON LA SOLUCION PROVISTA -----
Líneas iguales: 297
Total líneas: 300
Porcentaje: 99.00%
```

La librería propuesta por Python tiene un acierto del 99% de los casos, fallando solamente en 3 líneas. Este será utilizado como segundo estándar de control para los dos métodos a probar.

Método 1 – Distribución de la frecuencia de las letras:

```
Resultados - Distribucion de la frecuencia de las letras
---- COMPARACION CON LA SOLUCION PROVISTA -----
Líneas iguales: 244
Total líneas: 300
Porcentaje: 81.33%
---- COMPARACION CON LANGDETECT -----
Líneas iguales: 243
Total líneas: 300
Porcentaje: 81.00%
```



Método 2 – Probabilidad de letra precedente

```
Resultados - Probabilidad de letras precedentes
---- COMPARACION CON LA SOLUCION PROVISTA -----
Líneas iguales: 282
Total líneas: 300
Porcentaje: 94.00%
---- COMPARACION CON LANGDETECT -----
Líneas iguales: 281
Total líneas: 300
Porcentaje: 93.67%
```

Como puede verse, el segundo método es mejor que el primero para identificar idiomas por 40 líneas más, acercándose el segundo a un porcentaje de acierto del 94%, aumentando la precisión en un 13% respecto al método 1.

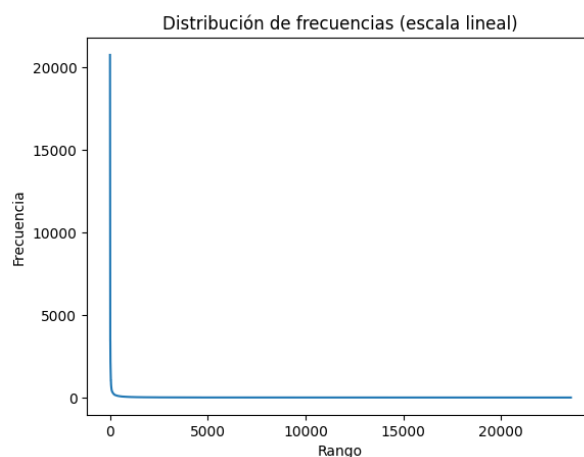
El método 2 es tan sólido que logra un 93.67% de similitud de comportamiento frente a la herramienta que provee Python

Utilización de IA:

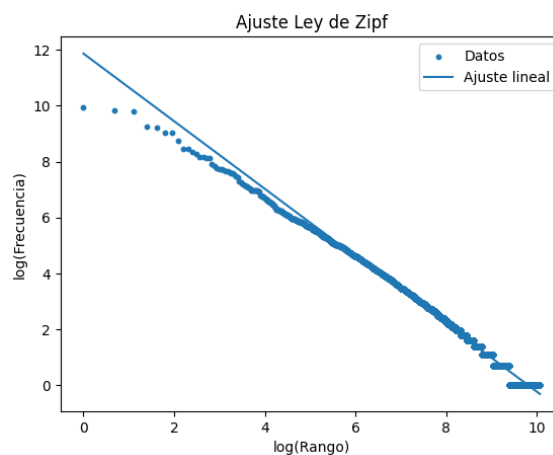
En este caso, la IA sirvió para la construcción de ambos métodos, con una explicación detallada del paso a paso para comprender de la mejor manera ambos algoritmos.

- 7) En este ejercicio se propone verificar la predicción de ley de Zipf. Para ello, descargue desde Project Gutenberg el texto del Quijote de Cervantes y escriba un programa que extraiga los términos y calcule sus frecuencias (el programa debe generar la lista ordenada por frecuencia descendente). Calcule la curva de ajuste utilizando la función *Polyfit* del módulo NumPy. Con los datos crudos y los estimados grafique en la notebook ambas distribuciones (haga 2 gráficos, uno en escala lineal y otro en log-log). ¿Cómo se comporta la predicción? ¿Qué conclusiones puede obtener?

Distribución en escala lineal



Distribución en escala log-log



A través del gráfico en escala log-log, podemos ver como la predicción se ajusta a una línea recta en lo que son el centro de los datos, que coinciden con las palabras que no son ni muy frecuente ni poco comunes.



En este caso, no realizamos la eliminación de stopwords, y eso puede verse en el gráfico de la escala lineal, donde la gran cantidad de frecuencia de palabras se encuentra al principio. Obteniendo los 10 términos más frecuente, tenemos esto:

1. que: 20769
2. de: 18411
3. y: 18272
4. la: 10492
5. a: 9934
6. en: 8285
7. el: 8267
8. no: 6356
9. los: 4769
10. se: 4752

Todas estas son palabras vacías, que sería necesario eliminar para hacer un correcto análisis.

De esta forma, a través del texto del Quijote de Cervantes, obra más destacada de la literatura española, demostramos que la ley de Zypf se cumple en todos los textos literarios.

- 8) Usando los datos del ejercicio anterior y de acuerdo a la ley de Zipf, calcule la cantidad de palabras que debería haber en el 10%, 20% y 30% del vocabulario. Verifique respecto de los valores reales. Utilice esta aproximación para podar el vocabulario en los mismos porcentajes e indique qué porcentaje de la poda coincide con palabras vacías. Extraiga las palabras podadas que no son *stopwords* y verifique si, a su criterio, pueden ser importantes para la recuperación.

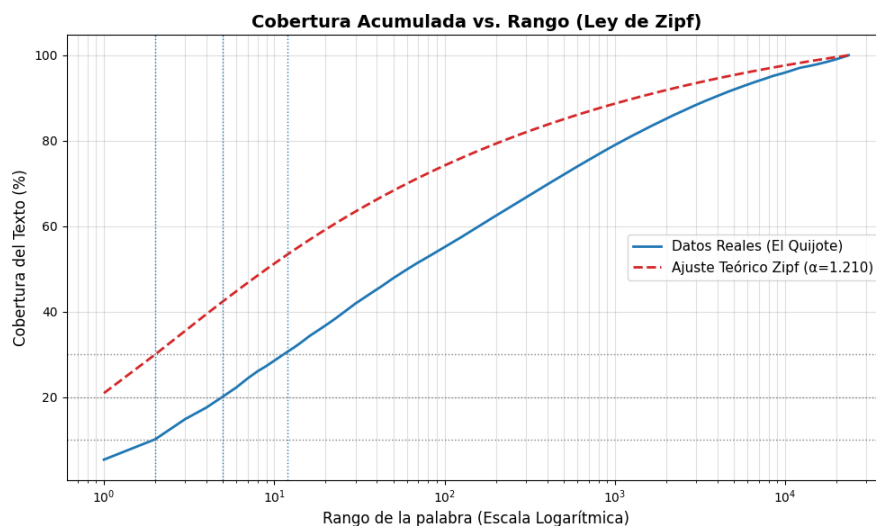
Cobertura Real vs. Teórica

Calculamos la frecuencia acumulada real y la teórica de Zipf, obteniendo lo siguiente:

Umbral	Real	Teórica (Zipf)	Diferencia
10%	2	1	+1
20%	5	1	+4
30%	12	3	+9

Con dos palabras es posible cubrir el 10% del documento. Analizando las más frecuentes, están dos palabras son “que” y “de”, con más de 38000 ocurrencias.

El modelo teórico dice que es necesaria una menor cantidad de palabras para representar esos % del vocabulario, subestimando al real.



Puede verse en el gráfico como la realidad se separa de la teoría en gran parte de los umbrales, llegando a converger casi sobre el final de este.

Poda del vocabulario

Realizamos la poda del vocabulario en los mismos umbrales utilizados anteriormente, obteniendo lo siguiente:



Umbral	Total	Stopwords	No-Stopwords
10%	2	2	0
20%	5	5	0
30%	12	12	0
35%	18	17	1

Del 30% del umbral, todas las palabras podadas fueron stopwords. Estas son las 12 primeras más frecuentes, que son: “que”, “de”, “y”, “la”, “a”, “en”, “el”, “no”, “los”, “se”, “con”, “por”. Un tercio del libro del Quijote son 100% stopwords.

Entre las 100 palabras mas frecuentes, 33 no son stopwords. Algunas de ellas son:

- Posición 17: Don
- Posición 21: Quijote
- Posición 22: Sancho
- Posición 26: Si

La primera no-stopword aparece en la posición 17, que sería casi el 35% del umbral.

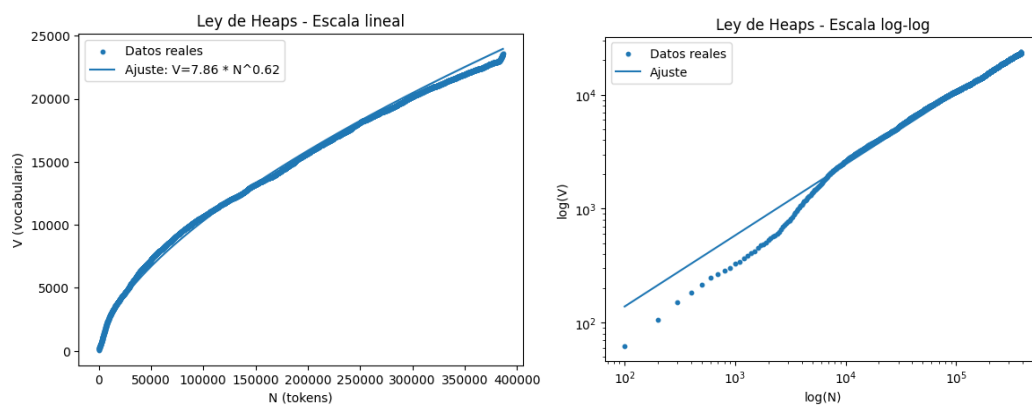
Como conclusión, podemos decir que esta poda resulta beneficiosa para un sistema de RI, ya que estamos eliminando palabras vacías, reduciendo un tercio del volumen del texto procesado sin perder ningún término semántico de interés.

Sin embargo, es necesario tener en cuenta el trade-off entre la poda del vocabulario por frecuencia, ya que esta puede mejorar la eficiencia y reducir el ruido, pero afectar la recuperación de la información eliminando términos relevantes en la obra.

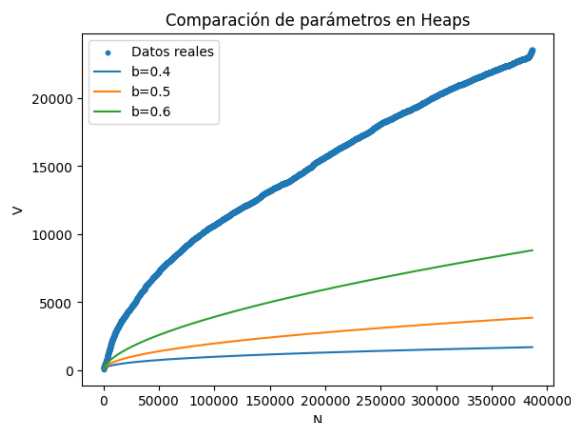
- 9) Codifique un script que reciba como parámetro el nombre de un archivo de texto, tokenize y calcule y escriba a un archivo los pares (#términos totales procesados, #términos únicos). Verifique en qué medida satisface la ley de Heaps. Grafique en la notebook los ajustes variando los parámetros de la expresión. Puede inicialmente probar con los archivos de los puntos anteriores.

Utilizando como archivo de texto nuevamente el Quijote de Cervantes, se realizó la construcción de los siguientes gráficos a través de los pares terminos totales – terminos unicos

Gráficos lineales y log-log para la ley de Heaps



Como puede verse en la escala lineal, a medida que crece el corpus, se incrementa el número de términos al principio, pero luego se hace más lento. Esto nos indica a la velocidad que crece el vocabulario, donde la aparición de términos nuevos se vuelve logarítmico en cierto punto, hasta llegar a un punto casi asintótico.



En este caso, el valor de B es 0,63 (rango 0.4 a 0.6 valido para verificar la ley de Heaps) aproximadamente, indicando que el texto es más diverso en cuanto a vocabulario, provocando que la curva no se vuelva asintótica con tanta rapidez. La elección del archivo del texto fue condicionante en este caso, ya que la obra es rica en vocabulario, comparado con Bs más chicos, donde se vuelve asintótica rápidamente, ya que es más repetitivo.



Bibliografía

- [1] Ricardo Baeza-Yates and Berthier Ribeiro-Neto. Modern Information Retrieval: The Concepts and Technology Behind Search. Addison-Wesley Publishing, USA, 2nd edition, 2008. Cap. 7.
- [2] Gregory Grefenstette and Pasi Tapanainen. What is a word, what is a sentence? problems of tokenization. In Rank Xerox Research Centre, pages 79–87, 1994.
- [3] Le Quan Ha, Darryl Stewart, Philip Hanna, and F Smith. Zipf and type-token rules for the english, spanish, irish and latin languages. Web Journal of Formal, Computational and Cognitive Linguistics, 1 (8):1–12, 1 2006.
- [4] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. Introduction to Information Retrieval. Cambridge University Press, New York, NY, USA, 2008. Cap.6.
- [5] Gabriel Tolosa, Fernando Bordignon. Introducción a la Recuperación de Información. Conceptos, modelos y algoritmos básicos. Laboratorio de Redes de Datos. UNLu, 2005. Cap. 3.
- [6] Daniel Jurafsky & James H. Martin. Speech and Language Processing. January 6, 2026. Cap. 2.